UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
PRIMER SEMESTRE 2026
LAB LENGUAJES FORMALES
Sección A

Manual técnico- Sistema de Análisis Académico

Nombre: Luis Armando Gaitán Lazáro
Carnet: 202404079
Fecha: 03/03/2026

Manual técnico- Sistema de Análisis Académico

Estructura del Programa

```
202404079_LFPA_1S2026/
│
├── main.cpp              # Archivo principal del sistema
├── estudiantes.csv       # Archivo de estudiantes
├── cursos.csv            # Archivo de cursos
├── notas.csv             # Archivo de notas
├── reportes/
│   ├── reporte1.html
│   ├── reporte2.html
│   ├── reporte3.html
│   ├── reporte4.html
│   └── reporte5.html
└── documentación/
    ├── ManualTecnico.pdf
    ├── ManualUsuario.pdf
    └── DiagramaFlujo.png
```

```
SISTEMA DE ANALISIS ACADEMICO
1. Cargar archivo de estudiantes
2. Cargar archivo de cursos
3. Cargar archivo de notas
4. Generar Reporte: Estadisticas por Curso
5. Generar Reporte: Rendimiento por Estudiante
6. Generar Reporte: Top 10 Mejores Estudiantes
7. Generar Reporte: Cursos con Mayor Reprobacion
8. Generar Reporte: Analisis por Carrera
9. Salir
Seleccione una opcion:
```

Software Requerido

Compilador: g++ (C++17 o superior)

Sistema Operativo: Windows / Linux

Editor recomendado: VS Code o CodeBlocks

Comando de compilación:

g++ -std=c++17 main.cpp -o sistema.exe

Hardware Mínimo

Procesador: 1 GHz
RAM: 2 GB
Espacio disco: 500 MB

Arquitectura del Sistema

El sistema está desarrollado en C++ usando programación estructurada y manejo de estructuras (struct) junto con vector para almacenamiento dinámico.

Se divide en:

Lectura de archivos
Procesamiento estadístico
Generación de reportes HTML
Menú interactivo en consola

Estructuras Principales

```
struct Estudiante {
    string carnet;
    string nombre;
    string carrera;
```

```
    int semestre;
};

struct Curso {
    string codigo;
    string nombre;
    int creditos;
};

struct Nota {
    string carnet;
    string codigoCurso;
    double nota;
};
```

Funciones Principales

Carga de Archivos

```
void cargarEstudiantes(string ruta);
void cargarCursos(string ruta);
void cargarNotas(string ruta);
```

Utilizan getline() y split(',') para lectura de archivos delimitados.

Cálculos Estadísticos

```
double calcularPromedio(vector<double>);
double calcularMediana(vector<double>);
double calcularDesviacion(vector<double>);
```

Generación de Reportes

```
void generarReporte1HTML();
void generarReporte2HTML();
void generarReporte3HTML();
void generarReporte4HTML();
void generarReporte5HTML();
```

Menú Principal

```
void mostrarMenu();
```

Permite:

Cargar estudiantes

Cargar cursos
Cargar notas
Generar reporte 1
Generar reporte 2
Generar reporte 3
Generar reporte 4
Generar reporte 5
Salir

Manejo de Excepciones

Verificación de apertura de archivos
Validación de vectores vacíos antes de cálculos
Control básico de errores en menú

```
if (!archivo.is_open()) {
    cout << "Error al abrir archivo" << endl;
}
```

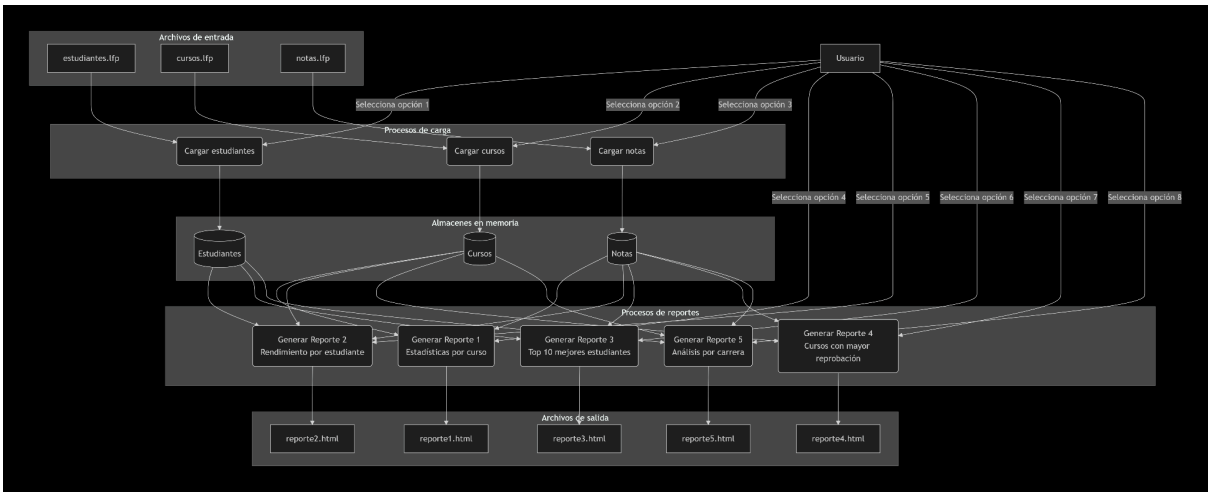
Límites del Sistema

La cantidad de datos depende de la memoria disponible.
Manejo dinámico con vector, sin límite fijo.
No se utiliza base de datos externa.

Flujo General del Sistema

Usuario carga archivos
Sistema almacena información en memoria
Usuario selecciona reporte
Sistema procesa estadísticas
Se genera archivo HTML
Usuario abre reporte en navegador

Flujo de datos


Archivos de entrada
estudiantes.lfp
cursos.lfp
notas.lfp
Usuario
Selecciona opción 1
Selecciona opción 2
Selecciona opción 3
Procesos de carga
Cargar estudiantes
Cargar cursos
Cargar notas
Selecciona opción 4
Selecciona opción 5
Selecciona opción 6
Selecciona opción 7
Selecciona opción 8
Almacenes en memoria
Estudiantes
Cursos
Notas
Procesos de reportes
Generar Reporte 2
Rendimiento por estudiante
Generar Reporte 1
Estadísticas por curso
Generar Reporte 3
Top 10 mejores estudiantes
Generar Reporte 5
Análisis por carrera
Generar Reporte 4
Cursos con mayor
reprobación
Archivos de salida
reporte2.html
reporte1.html
reporte3.html
reporte5.html
reporte4.html